



TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES WEB

Departamento de Informática

PROYECTO

Scribe - Plataforma E-commerce de Bolígrafos Artesanales

Manual Técnico

Índice

1. Introducción.....	3
2. Arquitectura de la aplicación.....	4
2.1 Frontend.....	4
2.1.1 Tecnologías usadas.....	4
2.1.2 Entorno de desarrollo.....	5
2.2 Backend.....	5
2.2.1 Tecnologías usadas.....	5
2.2.2 Entorno de desarrollo.....	6
2.3 Base de datos.....	6
2.3.1 Tecnologías usadas.....	6
2.3.2 Entorno de desarrollo.....	6
3. Documentación técnica.....	7
3.1. Análisis.....	7
3.2. Desarrollo (diagrama de secuencias,...).....	9
3.3. Pruebas realizadas.....	12
4. Proceso de despliegue.....	17
5. Propuestas de mejoras.....	19
6. Bibliografía.....	19

1. Introducción

La documentación del proyecto debe ajustarse a esta plantilla teniendo en cuenta los siguientes requisitos:

- **Formato del documento:** pdf.
- **Tipo de letra:** Arial.
- **Tamaño de letra:** 18pt para títulos, 16pt para subtítulos y 12pt para texto normal.
- **Texto justificado.**
- **La misma plantilla se puede utilizar para el Manual de Usuario.**

Se deben explicar las funcionalidades que se han desarrollado.

Scribe es una plataforma e-commerce especializada en la venta online de bolígrafos, plumas y portaminas artesanales fabricados a mano. El proyecto nace de la necesidad de digitalizar y profesionalizar la comercialización de la marca Scribe, que hasta el momento realizaba sus ventas de forma limitada a través de redes sociales y contacto directo, careciendo así de un sistema estructurado de gestión de pedidos e inventario.

El objetivo principal del proyecto es desarrollar una plataforma web funcional y profesional que automatice los procesos de venta, mejore la experiencia de compra del cliente y sirva de base para el crecimiento del negocio. La aplicación está dirigida a coleccionistas, profesionales y amantes de la escritura que buscan piezas únicas y de alto valor.

Respecto a las funcionalidades implementadas en la aplicación, encontramos las siguientes:

Catálogo de productos: visualización de bolígrafos artesanales con filtros por colección, material, precio y disponibilidad.

Sistema de usuarios: registro, login y gestión de perfiles para clientes y administrador.

Carrito de compra: añadir/eliminar productos, modificar cantidades y calcular totales.

Gestión de pedidos: registro de compras, seguimiento del estado del pedido y datos de envío.

Panel de administración: CRUD completo de productos, gestión de inventario y visualización de pedidos.

Fichas de producto: información detallada con galería de imágenes, descripción, precio y stock disponible.

2. Arquitectura de la aplicación

Se debe detallar qué tecnologías (base de datos, lenguaje de programación, frameworks, herramientas para el desarrollo de pruebas,...) se han usado para el desarrollo de cada parte de la aplicación.

Así mismo se deberán indicar los entornos de desarrollo usados.

En ambos casos se debe justificar la elección realizada e indicar las versiones de cada una de las herramientas usadas.

Se pueden incluir nuevos apartados si se considera necesario.

Mostrar la estructura del proyecto desarrollado.

La aplicación tiene una arquitectura cliente-servidor con separación clara entre el frontend (SPA con React) y el backend (API REST con Laravel). La comunicación entre ambas capas se realiza mediante peticiones HTTP con intercambio de datos en formato JSON.

2.1 Frontend

2.1.1 Tecnologías usadas

HTML5: Estructura semántica de la aplicación.

CSS3: Estilos y diseño responsive.

JavaScript ES2022: Interactividad y lógica de cliente.

React 19: Librería para construcción de interfaces dinámicas y gestión de estado.

npm 11.12.1: Gestor de paquetes para dependencias del frontend.

React fue elegido para el proyecto por su modelo de componentes reutilizables, su amplio ecosistema y por facilitar la creación de interfaces reactivas que mejoran notablemente la experiencia de usuario en un e-commerce donde el catálogo y el carrito requieren actualizaciones dinámicas sin recargar la página.

La guía de estilos del proyecto se define los siguientes valores:

Paleta de colores:

Beige claro #F5F1E8: fondos y backgrounds

Beige medio #D4C5B0: secciones y cards

Beige oscuro/marrón #8B7355 — botones, títulos y footer

Negro/gris oscuro #2C2C2C — textos principales

Dorado suave #C9A86A — acentos, hover y detalles premium

Blanco #FFFFFF — contraste y textos sobre fondos oscuros

Tipografías:

Títulos: Playfair Display (serif elegante)

Cuerpo de texto: Inter (sans-serif legible)

2.1.2 Entorno de desarrollo

Sistema Operativo: Mac OS

IDE: Visual Studio Code (VS Code)

Servidor local: XAMPP v7.4.33

Navegadores de prueba: Google Chrome, Mozilla Firefox, Safari

2.2 Backend

2.2.1 Tecnologías usadas

PHP 8.4.6: Lenguaje de programación del servidor, maduro y ampliamente soportado

Laravel Framework 12.53.0: framework de backend. Se encarga de la API REST, base de datos, autenticación, etc.

MySQL 8.0: Sistema de gestión de bases de datos relacional, robusto y de código abierto.

Composer 2.8.8: Gestor de dependencias para PHP/Laravel.

Postman: Pruebas y documentación de endpoints de la API REST.

Laravel fue seleccionado para este proyecto por su arquitectura MVC bien definida, su ORM Eloquent que simplifica las operaciones con la base de datos, su sistema de migraciones para control de versiones del esquema y su soporte nativo para APIs REST mediante Laravel Sanctum para la autenticación.

2.2.2 Entorno de desarrollo

Sistema Operativo: Mac OS

IDE: Visual Studio Code (VS Code)

Servidor local: XAMPP v3.3.0 (Apache + MySQL)

Control de versiones: Git + GitHub

2.3 Base de datos

2.3.1 Tecnologías usadas

MySQL 8.0 es el sistema de gestión de bases de datos relacional que he utilizado en el proyecto. Se trata de un sistema maduro, ampliamente adoptado en aplicaciones web y de código abierto, lo que lo convierte en una elección muy buena para un proyecto e-commerce como Scripe. Su integración con Laravel a través del ORM Eloquent simplifica enormemente las operaciones de consulta, inserción y actualización de datos sin necesidad de escribir SQL manualmente.

La base de datos está formada por seis tablas relacionadas entre sí mediante claves foráneas: usuarios, direcciones, pedidos, contenido_pedidos, productos y categorías. El diseño sigue los principios de normalización para evitar redundancia de datos y facilitar el mantenimiento del esquema.

2.3.2 Entorno de desarrollo

El servidor MySQL se gestiona localmente a través de XAMPP v3.3.0, que incluye el motor MySQL y la herramienta phpMyAdmin para la administración visual de la base de datos. En producción, la base de datos está desplegada como servicio independiente en Railway, conectada al backend Laravel mediante variables de entorno.

3. Documentación técnica

3.1. Análisis

Incluir los diagramas que se hayan desarrollado para el desarrollo de la aplicación (diagramas de casos de uso, diagrama de clases, diagrama entidad relación,...).

Se puede incluir toda la documentación que se estime oportuna.

Casos de Uso

El sistema contempla tres tipos de actores: **Cliente registrado**, **Usuario invitado** y **Administrador**.

Actor: Usuario Invitado =>

- Navegar por el catálogo de productos
- Ver ficha detallada de producto
- Añadir productos al carrito (requiere registro para finalizar compra)
- Registrarse en el sistema

Actor: Cliente Registrado =>

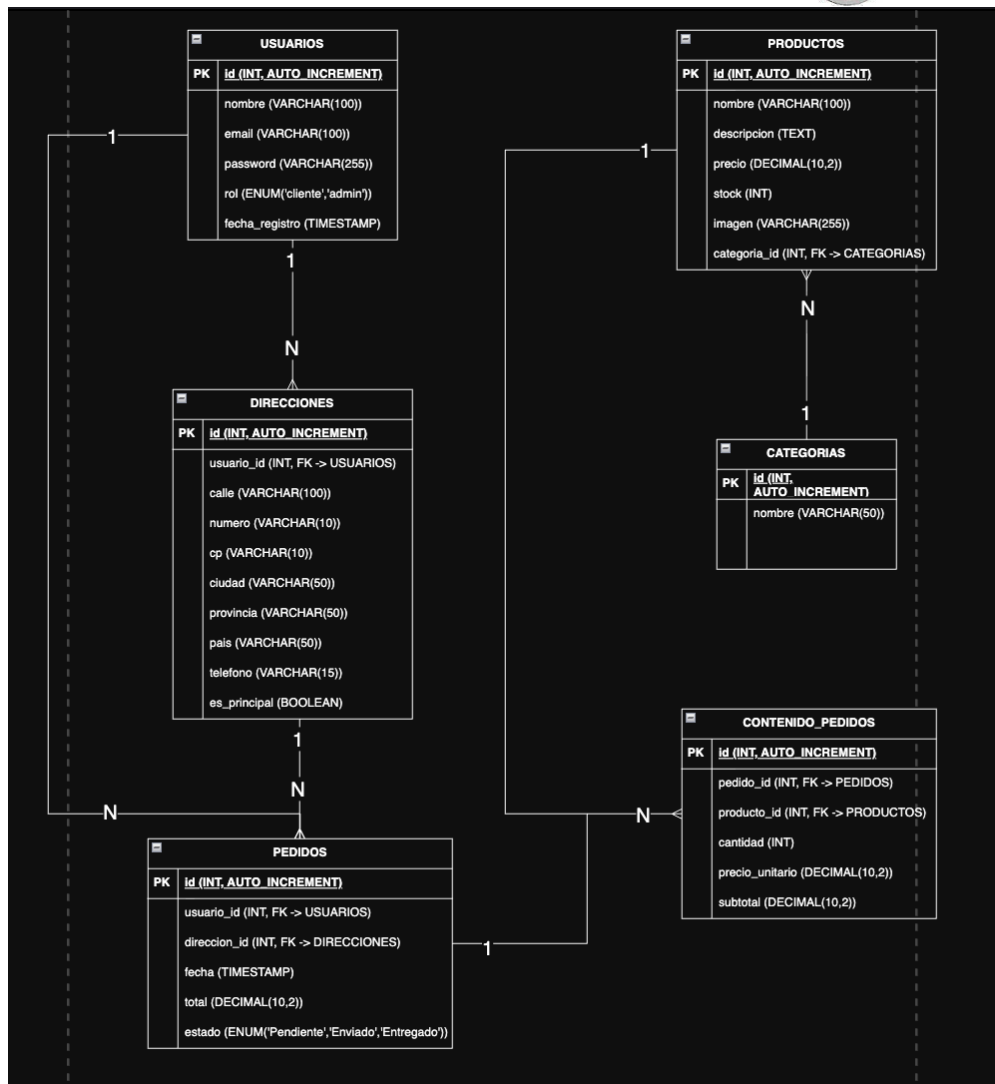
- Iniciar y cerrar sesión
- Gestionar perfil y direcciones de envío
- Realizar pedidos y proceso de checkout
- Consultar historial de pedidos

Actor: Administrador =>

- Gestionar productos (crear, editar, eliminar)
- Gestionar stock e inventario
- Visualizar y gestionar el estado de los pedidos
- Administrar colecciones y categorías

Diagrama Entidad-Relación

La base de datos relacional de Scripe está formada por seis tablas conectadas mediante claves foráneas:



Decisiones de diseño clave:

La tabla **CONTENIDO_PEDIDOS** actúa como tabla auxiliar para gestionar la relación N:M entre PEDIDOS y PRODUCTOS, almacenando así el precio unitario en el momento de la compra para poder guardar el historial aunque el precio del producto cambie después.

La tabla **DIRECCIONES** se separa de **PEDIDOS** para aplicar normalización, evitando así la redundancia y permitiendo que un usuario tenga múltiples direcciones guardadas. El campo booleano **es_principal** identifica la dirección por defecto.

El campo estado de **PEDIDOS** es de tipo **ENUM('Pendiente', 'Enviado', 'Entregado')**, representando el ciclo de vida básico de un pedido de manera simple y realista.

Diagrama de clases:

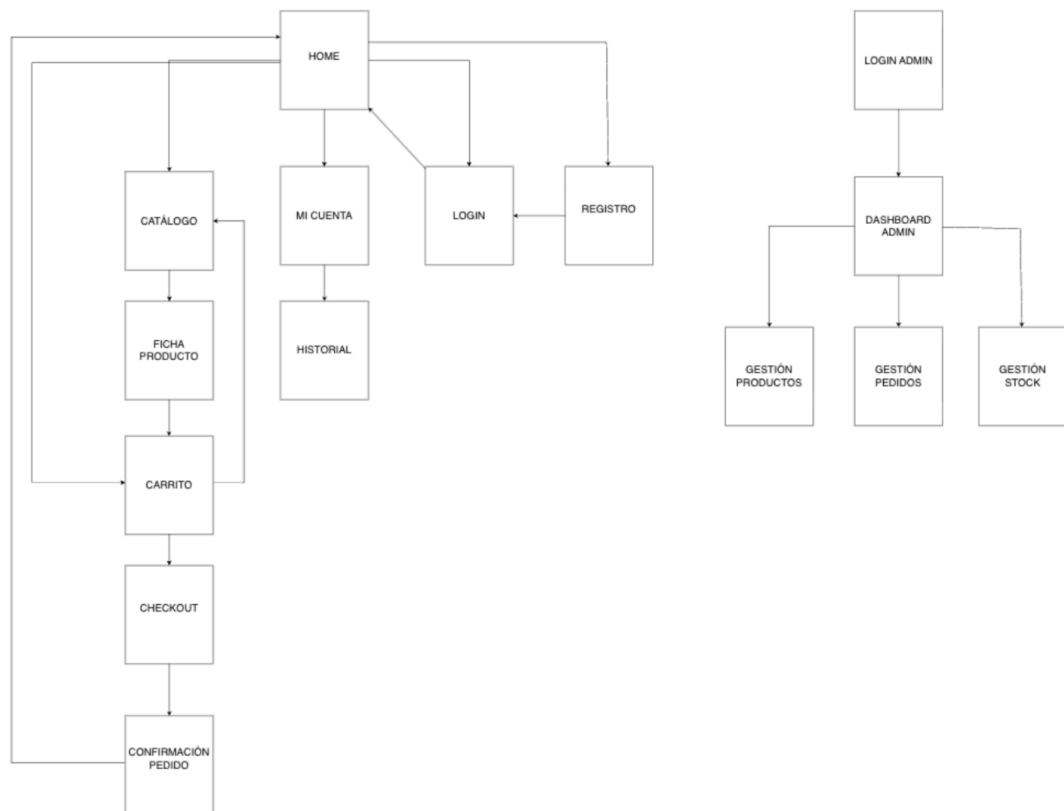


3.2. Desarrollo (diagrama de secuencias,...)

Incluir los diagramas que se hayan desarrollado durante el para el desarrollo de la aplicación (diagramas de secuencias, modelo lógico, ...).

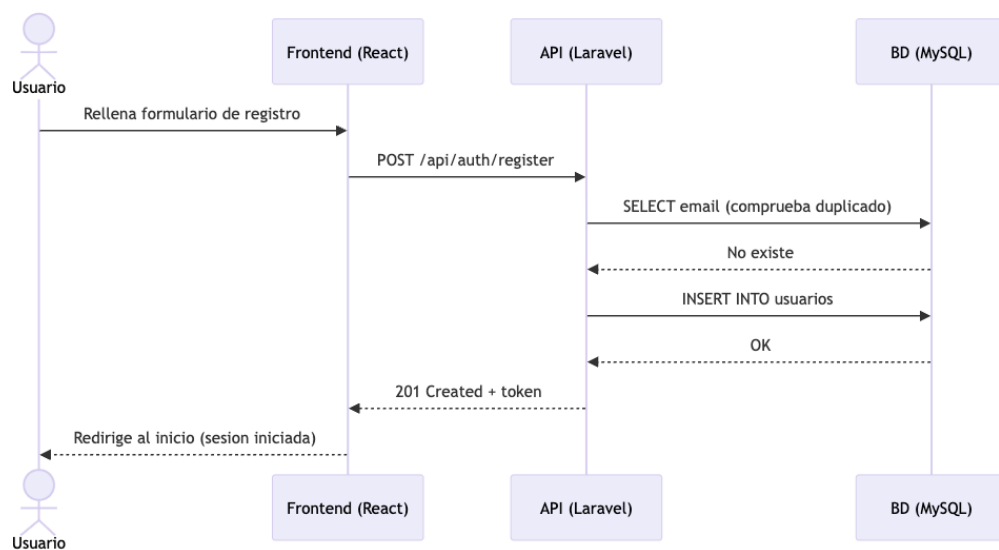
Se puede incluir toda la documentación que se estime oportuna.

Diagrama de flujo:

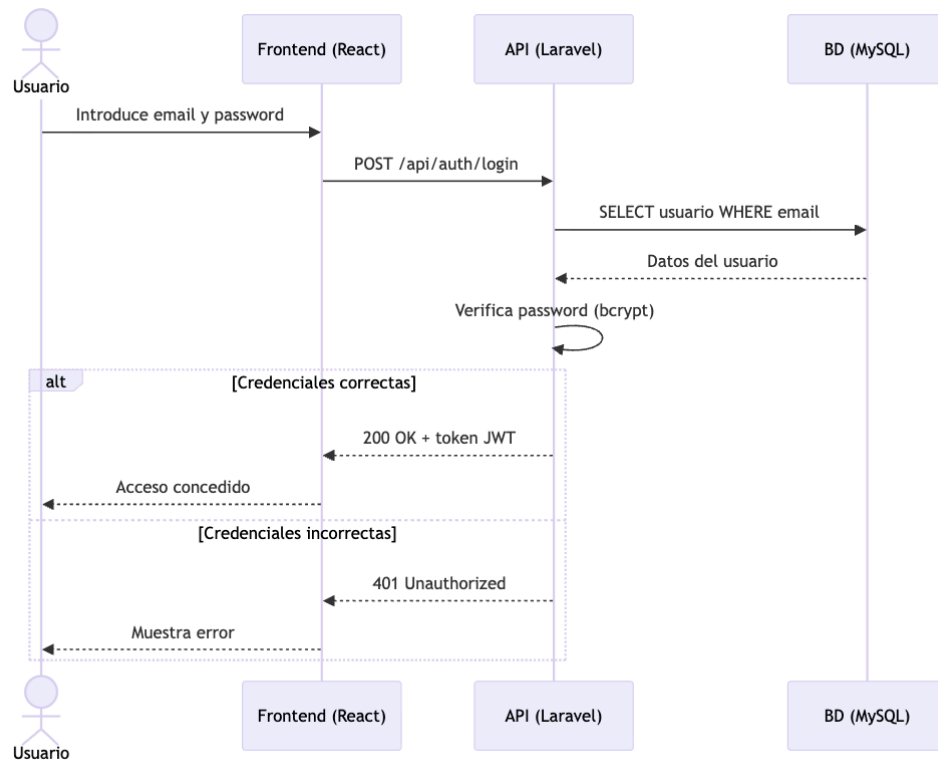


Diagramas de secuencias:

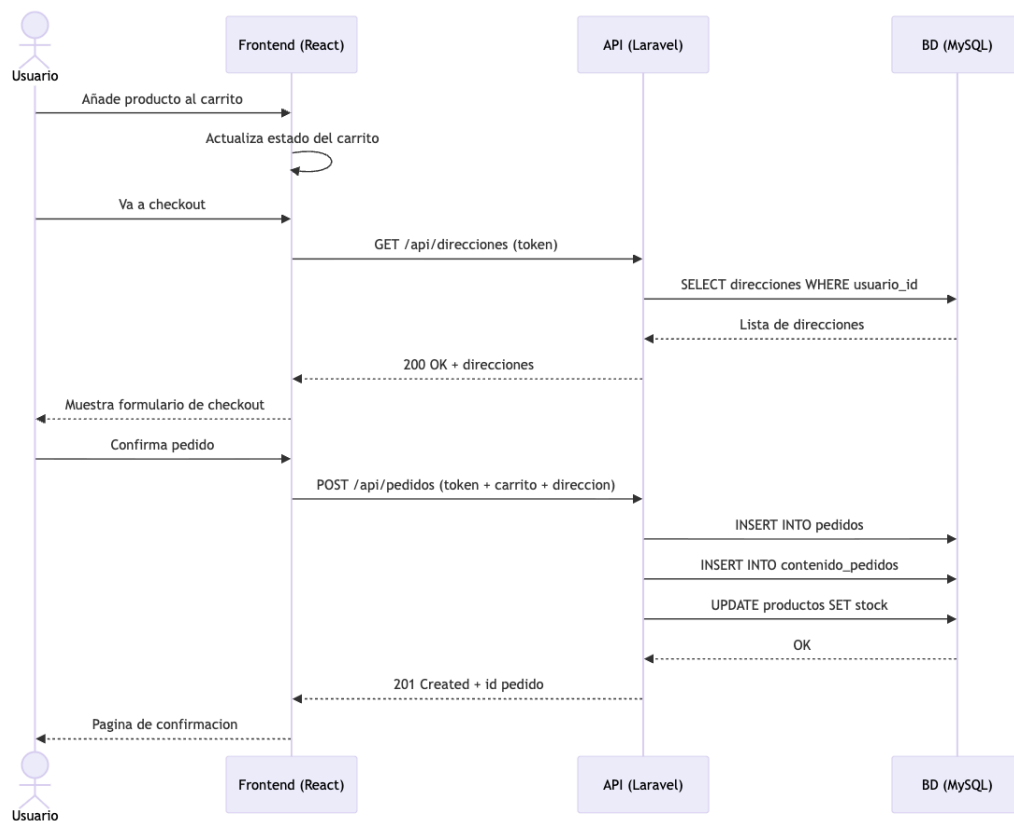
3.2.1 Registro de usuario



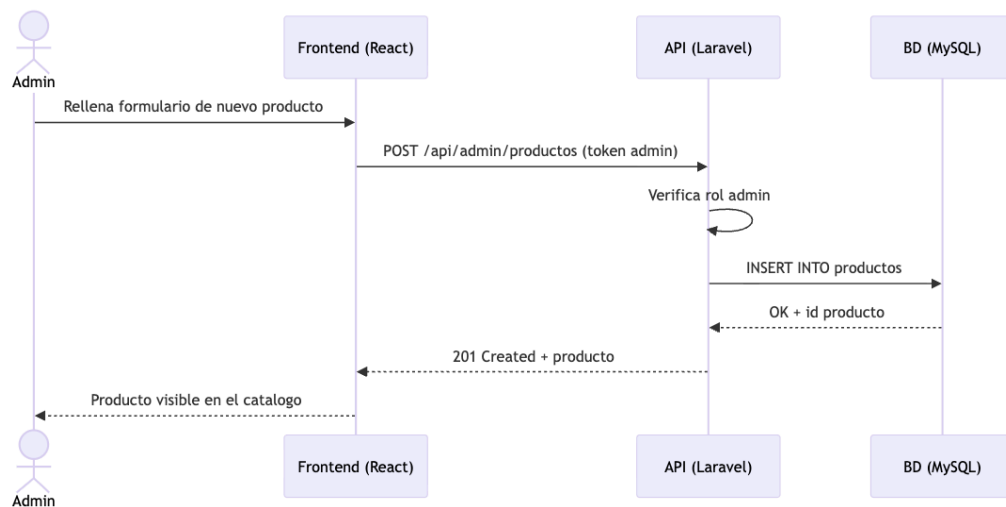
3.2.2 Login de usuario



3.2.3 Proceso de compra



3.2.4 Admin crea el producto



Los principales endpoints implementados en la API REST son los siguientes:

Autenticación: POST /api/auth/register para el registro de nuevos usuarios y POST /api/auth/login para la autenticación y obtención del token.

Productos: GET /api/productos para obtener el listado con filtros y GET /api/productos/{id} para el detalle de un producto concreto.

Pedidos: POST /api/pedidos para crear un nuevo pedido y GET /api/pedidos para consultar el historial del usuario.

Panel de administración: GET /api/admin/productos para listar productos, POST /api/admin/productos para crear uno nuevo, PUT /api/admin/productos/{id} para editarlo y DELETE /api/admin/productos/{id} para eliminarlo.

3.3. Pruebas realizadas

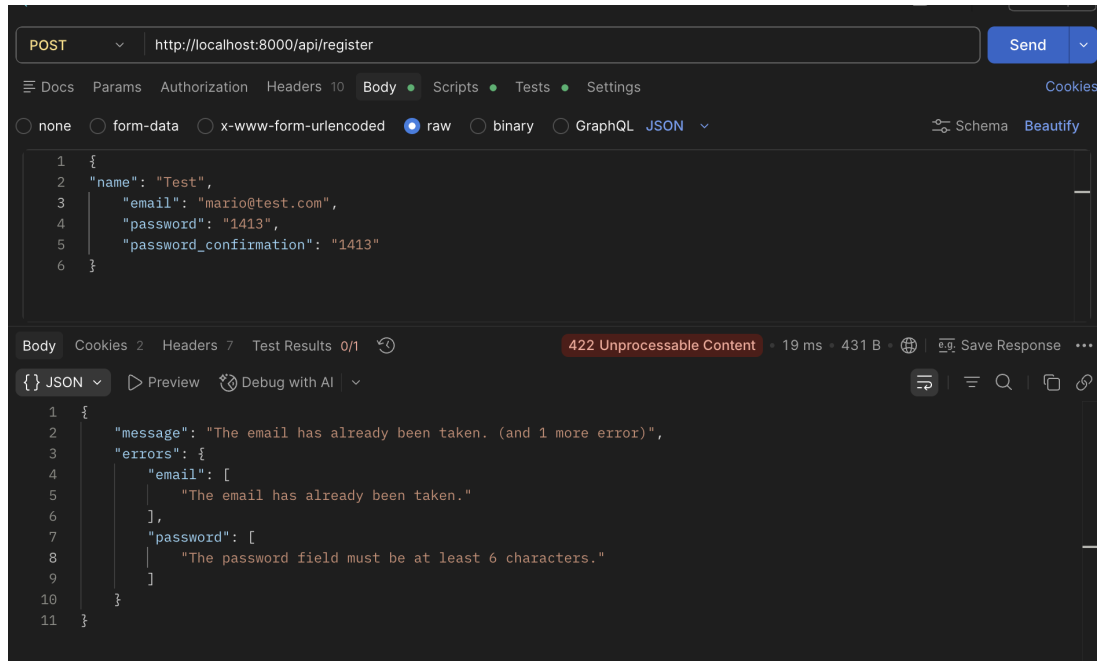
Detallar algunas de las pruebas que se han realizado indicando:

- **Componente (clase, método,...)** al que se le han realizado.
- **Juego de datos de entrada.**
- **Resultado de salida.**
- **Herramientas de pruebas usadas.**

Para las validaciones vamos a usar la aplicación **Postman**, que nos permite visualizar los procesos **GET** y **POST** de nuestra aplicación, pudiendo así comprobar que todo

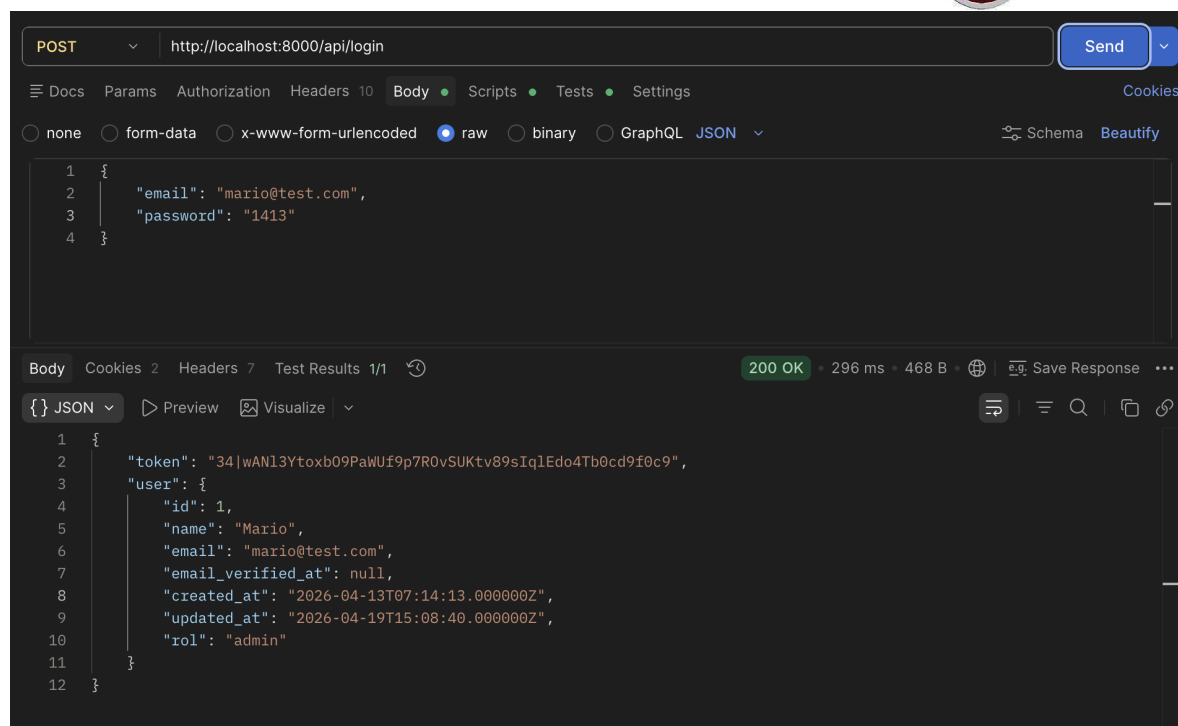
esté correcto.

Prueba 1 - Verificar login con e-mail incorrecto:



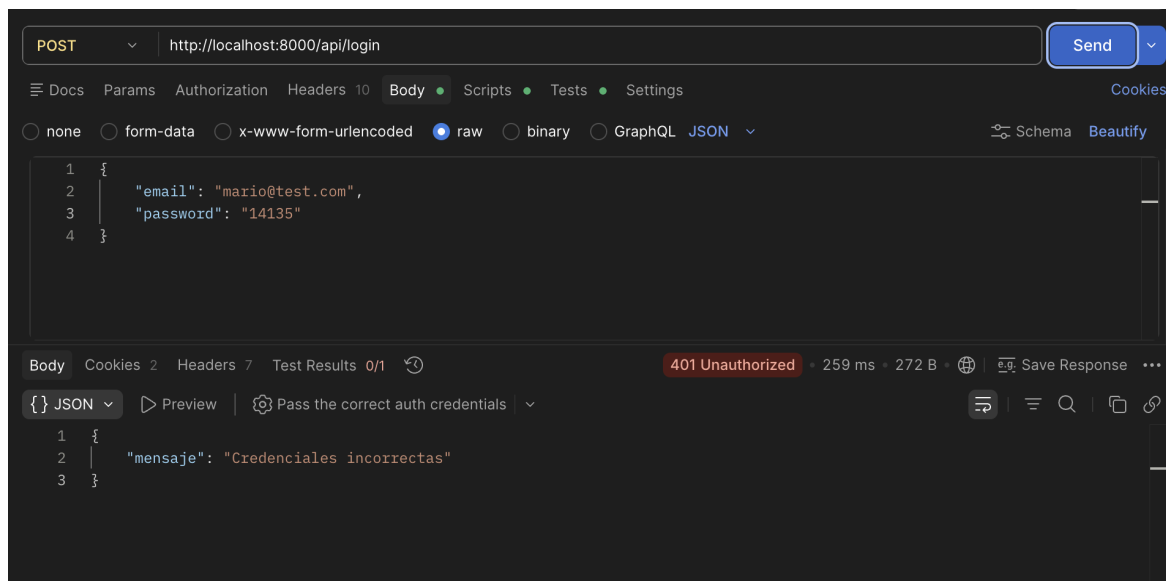
Como podemos apreciar en la prueba número uno con <http://localhost:8000/api/register> vemos que salta un error que es correcto, ya que se está usando un email que ya existe en el registro pero aún así podemos crear una contraseña que contenga menos de seis caracteres, aunque en el json nos muestre que no es viable. Esto es un error de producción que con herramientas como **Postman** podemos verificar y corregir posteriormente.

Prueba 2 - Login con credenciales correctas:



Como podemos apreciar en la imagen superior de la prueba dos con <http://localhost:8000/api/login> vemos como obtenemos un **200 OK** al introducir los datos del usuario ya creado, en este caso con **rol admin**. También podemos ver el **token** generado. Aún así, vemos que la contraseña es de cuatro dígitos, confirmando el suceso de la prueba uno.

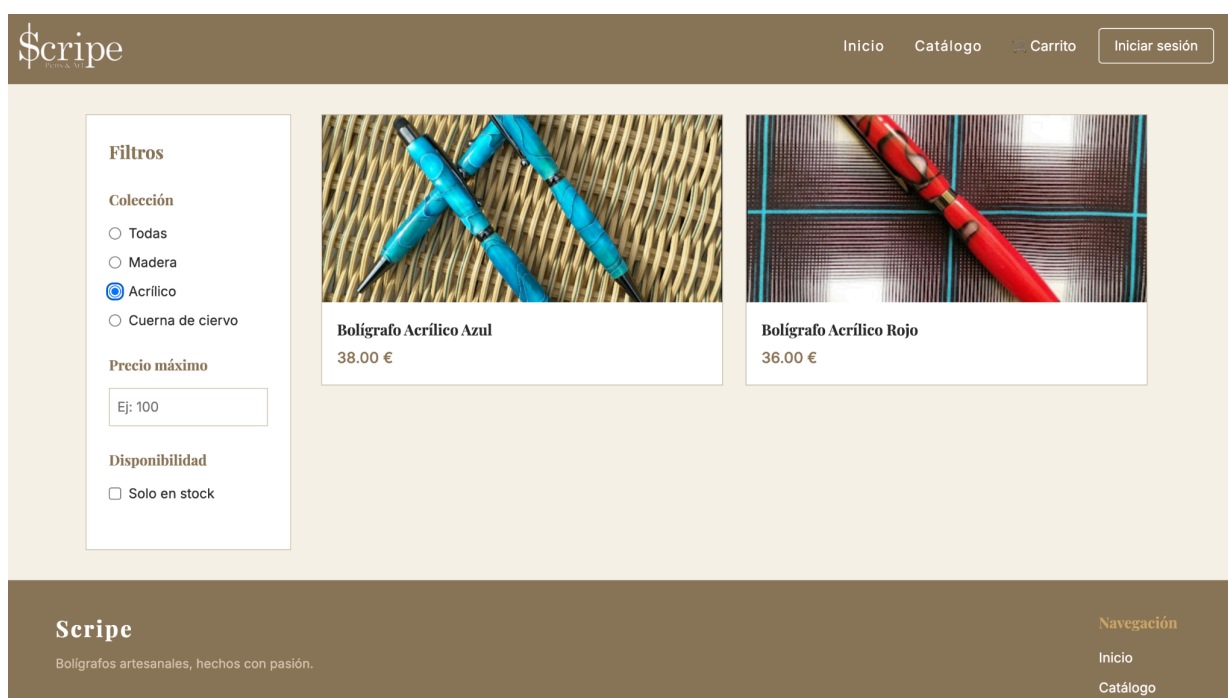
Prueba 3 - Login con contraseña incorrecta:



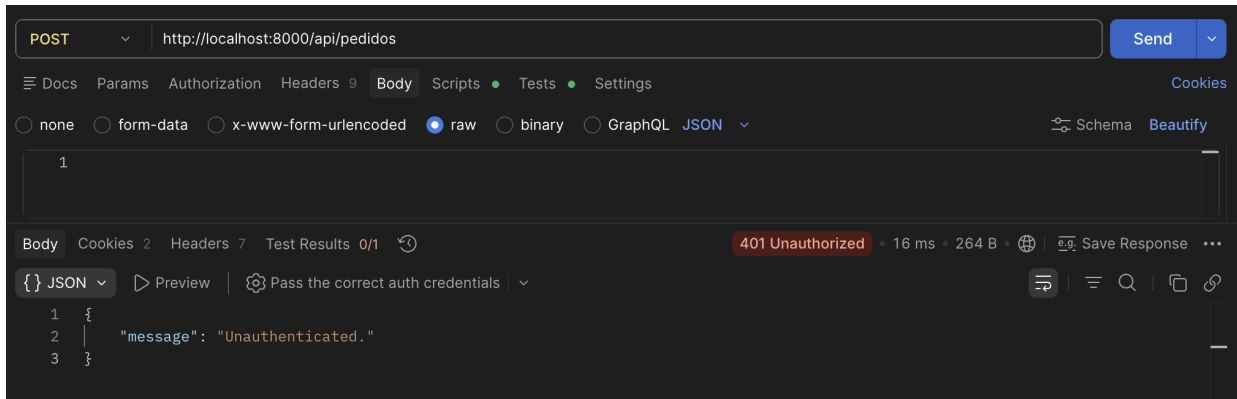
Como se puede apreciar en la imagen superior, en este caso se nos devuelve un **401 Unauthorized**, confirmando así que la validación es correcta para credenciales incorrectas.

Prueba 4 - Filtrar productos por categoría:

El filtrado de productos por categoría, precio y disponibilidad se realiza en el lado del cliente (React). El componente `Catalogo.jsx` obtiene todos los productos mediante `GET /api/productos` y aplica los filtros en cascada sobre el array usando el método `.filter()` de JavaScript.

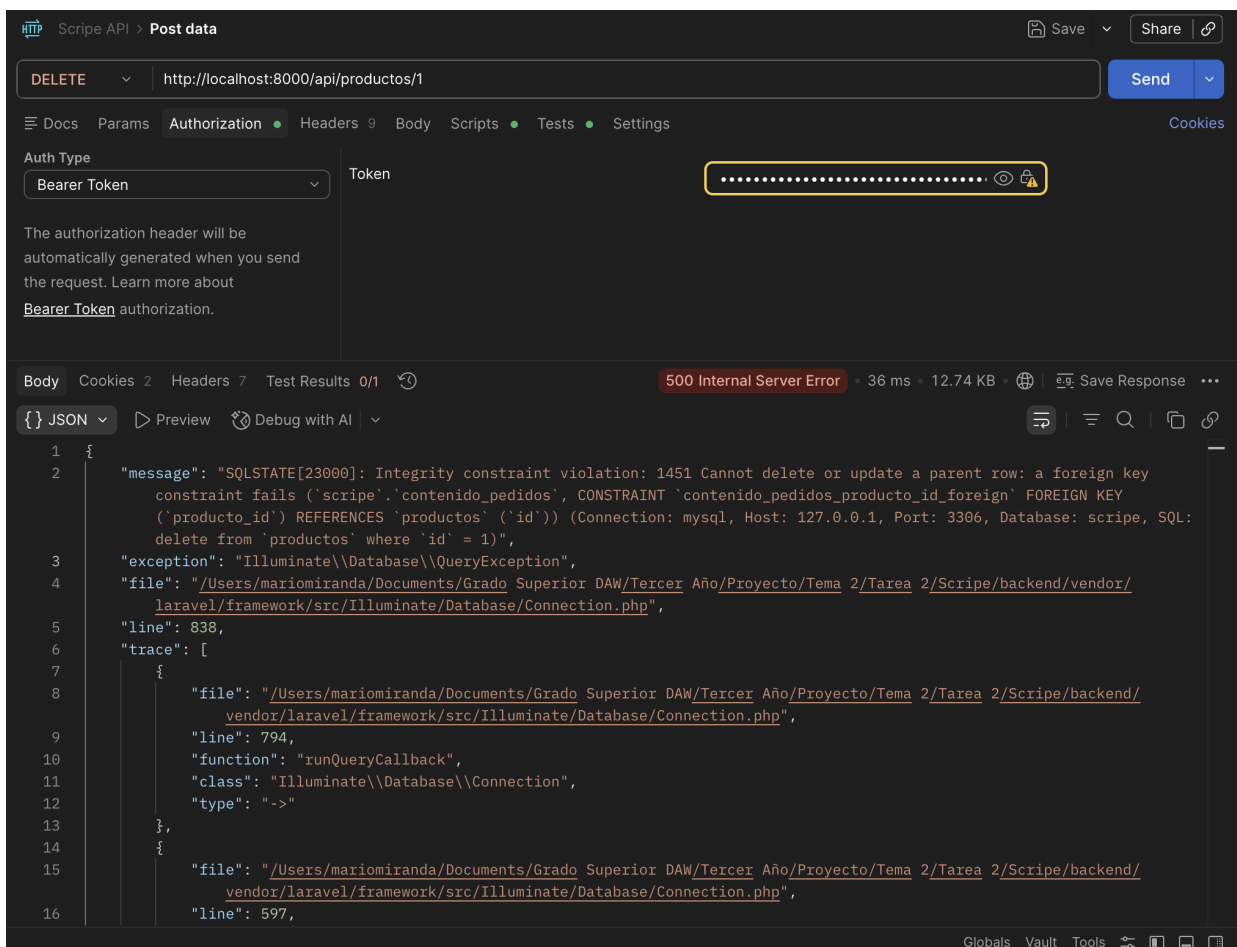


Prueba 5 - Crear pedido sin token:



Como se puede apreciar en la imagen superior, al querer crear un pedido sin token se nos devuelve el valor “**Unauthenticated**”, confirmando de esta manera que la aplicación devuelve un error si el token no existe.

Prueba 6 - Eliminar producto con token de cliente (no admin):



Como podemos ver en la imagen superior, Laravel da un error 500 e intenta borrar, ya que no devuelve un 403. La protección del panel admin se está realizando en el frontend, ya que **React** se encarga de ocultar el panel si el usuario no es **admin**, pero

a su vez el backend no comprueba el rol en las rutas.

Resultado esperado: 403 Forbidden

Resultado obtenido: La validación la realiza el front y el backend no verifica el rol en

4. Proceso de despliegue

El contenido de este apartado irá destinado a poder desplegar la aplicación y poder ejecutarla.

Si la aplicación requiere despliegue, se deberá explicar detalladamente:

- Software necesario (indicar versiones y repositorios para descarga) para poder desplegar la aplicación.
- Proceso detallado para el despliegue.

Si se ha usado un hosting:

- Detallar cual es y la forma de acceder a la aplicación.

En cualquier otro caso, incluir las informaciones necesarias para poder usar la aplicación.

Software necesario

Para desplegar la aplicación en local se necesita el siguiente software:

En primer lugar, **XAMPP v7.4.33**, que incluye Apache y MySQL, disponible en <https://www.apachefriends.org>. **PHP 8.4.6**, actualizado. También es necesario **Composer 2.8.8**, el gestor de dependencias de Laravel, descargable desde <https://getcomposer.org>. Para el frontend se necesita Node.js 18 LTS junto con npm, disponible en <https://nodejs.org>. Por último, **Git 2.50.1** para clonar el repositorio, descargable desde <https://git-scm.com>.

Proceso de despliegue en local

El primer paso es clonar el repositorio con git clone y acceder a la carpeta del proyecto. A continuación hay que iniciar XAMPP y arrancar los módulos Apache y MySQL desde su panel de control.

Para configurar el backend, acceder a la carpeta del backend y ejecutar composer install para instalar las dependencias. Después copiar el archivo .env.example como .env y ejecutar php artisan key:generate. En el .env configurar los datos de conexión a MySQL: host 127.0.0.1, puerto 3306, base de datos scribe_db, usuario root y

contraseña vacía por defecto en XAMPP.

Una vez configurado, ejecutar php artisan migrate para crear las tablas y php artisan db:seed para cargar los datos iniciales. Finalmente, levantar el backend con php artisan serve, quedando disponible en <http://localhost:8000>.

Para el frontend, acceder a la carpeta correspondiente, ejecutar npm install para instalar las dependencias y npm run dev para iniciar el servidor de desarrollo, disponible en <http://localhost:5173>.

Despliegue en producción (Railway)

La aplicación está desplegada en producción en Railway (<https://railway.app>), con las tres capas separadas en servicios independientes: el frontend React, el backend Laravel y la base de datos MySQL. Esta arquitectura permite escalar cada servicio de forma independiente y facilita el mantenimiento. Railway gestiona automáticamente las variables de entorno y la conexión entre servicios.

Acceso a la aplicación

La aplicación en producción es accesible directamente desde el navegador sin necesidad de instalar ningún software, a través de la siguiente URL:

<https://scripefront-production.up.railway.app/>

5. Propuestas de mejoras

Detallar todas las mejoras que se podrían realizar sobre la aplicación desarrollada.

Protección de rutas de administración en el backend: la restricción de acceso al panel de administración se gestiona actualmente solo desde el frontend. Se propone implementar un middleware de verificación de rol en Laravel que proteja las rutas de la API, devolviendo así un error 403 a cualquier petición que no provenga de un usuario con rol administrador.

Pasarela de pago real: el proceso de pago actual es una simulación. Se propone integrar una pasarela de pago real como Stripe o PayPal, permitiendo a los clientes realizar pagos seguros con tarjeta u otros métodos directamente desde la plataforma.

Filtrado de productos en el backend: Actualmente el filtrado por categoría, precio y disponibilidad se realiza en el frontend mediante JavaScript. Se propone migrar esta lógica al backend implementando parámetros query en la API REST. De esta manera la base de datos devuelve únicamente los productos necesarios, mejorando el rendimiento con catálogos grandes.

Generación automática de facturas: Se propone implementar la generación y envío automático de una factura en formato PDF al correo electrónico del cliente una vez confirmado el pedido, utilizando librerías como Laravel DomPDF.

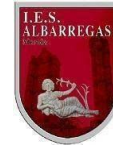
Sistema de reseñas y valoraciones: Se propone añadir la posibilidad de que los clientes que hayan realizado una compra puedan dejar valoraciones y comentarios sobre los productos, aumentando la confianza de otros compradores y mejorando la experiencia de compra.

6. Bibliografía

Se deben incluir la referencias a todos los recursos utilizados en el desarrollo del proyecto, tanto libros, manuales, etc como bibliografía web.

Documentación Oficial

- Documentación oficial de React. Meta Open Source. <https://react.dev/>
- Documentación oficial de Laravel 12.53.0 Laravel LLC. <https://laravel.com/docs/10.x>
- Documentación oficial de MySQL 8.0. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- Documentación oficial de PHP 8.2. The PHP Group. <https://www.php.net/manual/es/>



Herramientas de Desarrollo

- XAMPP - Apache, MySQL, PHP y Perl. Apache Friends. <https://www.apachefriends.org/es/index.html>
- Postman API Platform. Postman Inc. <https://www.postman.com/>
- Visual Studio Code. Microsoft. <https://code.visualstudio.com/>
- Figma - Design Tool. Figma Inc. <https://www.figma.com/>

Recursos de Aprendizaje

- MDN Web Docs - Referencia de HTML, CSS y JavaScript. Mozilla Foundation. <https://developer.mozilla.org/es/>
- Laracasts - Video Tutorials for Laravel Developers. <https://laracasts.com/>
- W3Schools - Online Web Tutorials. <https://www.w3schools.com/>
- Stack Overflow - Developer community. <https://stackoverflow.com/>

Recursos de Diseño

- Google Fonts - Playfair Display e Inter. Google LLC. <https://fonts.google.com/>
- Guía de Accesibilidad WCAG 2.1. W3C. <https://www.w3.org/TR/WCAG21/>